# BotStudioX — AI Virtual Try-On Platform

### Professional Product & Technical Documentation

**Author:** Mohan,siva **Version:** 1.0 (Production)
**Last Updated:** April 2026

---

## Table of Contents

1. Executive Summary
2. Product Overview
3. System Input & Output Specification
4. Development Journey
5. Production Architecture
6. Pipeline — Step-by-Step Flow
7. Feature Modules
8. Technical Stack
9. Performance Benchmarks
10. Known Limitations
11. Glossary

---

## 1. Executive Summary

BotStudioX is an AI-powered virtual try-on and fashion image generation platform designed for e-commerce. It allows fashion brands and sellers to generate high-quality product imagery — showing a model wearing a specific garment, in a specific pose, against a specific background — without a physical photoshoot.

The system evolved from a simple 4-input prototype to a production-grade multi-stage AI pipeline powered by Google's Gemini models, with a full React-based web UI, storyboard management, print-on-garment compositing, multi-angle generation, and an asset library.

**Core Value Proposition:**
Replace expensive fashion photoshoots with AI-generated imagery that is consistent, customizable, and production-ready.

---

## 2. Product Overview

### What BotStudioX Does

BotStudioX takes a set of user-provided inputs (garment, model, pose, background) and produces a photorealistic composite image of a model wearing that garment in the given pose, placed in the given background environment.

### Key Capabilities

| Capability | Description |
|---|---|
| Virtual Try-On | Model wearing any uploaded garment |
| AI Look Planning | Auto-generates styling plan from garment image |
| Pose Control | Reference pose images or preset poses |

| | |
|---|---|
| Background Placement | Custom background images or themed presets |
| Print-on-Garment | Apply design prints onto garment templates (front / back / side) |
| Multi-Angle Views | Generate side and back views from a front composite |
| Storyboard System | Manage multiple garment shoots as named storyboards |
| Asset Library | Reusable model, pose, and background asset management |
| Retry / Correction | Regenerate with user feedback and corrections |

---

## 3. System Input & Output Specification

### Inputs

| Input | Type | Required | Max Count | Notes |
|---|---|---|---|---|
| Garment Image | Image file (JPG/PNG) | Yes | 4 | The garment to be worn |
| Model Image | Image file (JPG/PNG) | Yes | 4 | Identity reference (face, body) |
| Pose Image | Image file (JPG/PNG) | No | 4 | Body pose / skeleton reference |
| Background Image | Image file (JPG/PNG) | No | 4 | Scene/environment background |
| Occasion | Preset or text | No | 1 | e.g. Casual, Date Night, Festival |
| Style | Preset or text | No | 1 | e.g. Minimal, Streetwear, Luxe |
| Model Ethnicity | Preset or text | No | 1 | e.g. White/European, South Asian |
| Footwear | Preset or text | No | 1 | e.g. Sneakers, Heels, Sandals |
| Accessories | Free text | No | — | e.g. gold earrings, minimal watch |
| Model Styling Notes | Free text | No | — | e.g. sleek hair, natural glam makeup |

**Storage:**
Inputs are stored per storyboard using `localStorage` (config/metadata) and `IndexedDB` (image data URLs). Each field supports up to 4 images to allow asset variation and reuse across runs.

### Outputs

| Output | Format | Dimensions | Notes |
|---|---|---|---|
| Primary Composite | Image (PNG/JPEG) | 1080 × 1440 px (3:4) | Full-body, head-to-toe |
| Garment Reference | Image (PNG/JPEG) | Variable | Intermediate clean garment image |

| | | | |
|---|---|---|---|
| Multi-Angle — Side | Image (PNG/JPEG) | 1080 × 1440 px | Generated from composite |
| Multi-Angle — Back | Image (PNG/JPEG) | 1080 × 1440 px | Generated from composite |
| Print Composite — Front | Image (PNG/JPEG) | Variable | Design applied to garment front |
| Print Composite — Back | Image (PNG/JPEG) | Variable | Design applied to garment back |
| Print Composite — Side | Image (PNG/JPEG) | Variable | Design applied to garment side |

**All outputs** are available for immediate download or saved to the in-app gallery (IndexedDB).

---

## 4. Development Journey

The system was built iteratively over 9 phases, each solving specific technical problems.

### Phase 1 — Initial Concept

**Goal:** Prove the idea — can Gemini generate a model wearing a garment?
**Stack:** Streamlit + Gemini API
**Result:** Basic output generated. Pose was inaccurate; clothing alignment was inconsistent.

### Phase 2 — Problem Analysis

**Identified 3 core requirements:**

1. Pose Control — model must follow the reference pose
2. Identity Preservation — model face/body must not change
3. Garment Transfer — garment must be rendered faithfully on the model

**Key Realization:** Gemini is a generative model, not a conditioned diffusion model. Controlling it requires careful prompt engineering, not model fine-tuning.

### Phase 3 — First Working Pipeline

- Integrated Gemini API with multi-image prompting
- Built basic Streamlit UI for 4-input upload
- **Result:** ~50% pose accuracy; unstable across runs

### Phase 4 — Pose Signal Improvement (OpenCV + MediaPipe)

- Introduced OpenCV and MediaPipe for skeleton extraction
- Skeleton (black background + white joints) provided a cleaner pose signal
- **Result:** Pose accuracy improved to ~60–70%

### Phase 5 — Prompt Engineering Optimization

Two critical improvements:

1. **STRICT RULES** in prompt — repeated pose constraints, explicit negative rules
2. **Input ordering fix:**
    - Before: `[prompt, model, garment, pose, bg]`
    - After: `[prompt, pose, model, garment, bg]`
    - Placing pose first significantly improved Gemini's adherence to it

### Phase 6 — Dual-Mode System

Built two generation modes:

- **Advanced Mode** — uses OpenCV skeleton; higher pose accuracy
- **Gemini Only Mode** — uses raw pose image; faster but less precise

### Phase 7 — Critical Identity Bug

**Problem:** When preset pose images were used, the output showed the person from the pose image wearing the garment instead of the target model.
**Root Cause:** Gemini treated the pose image as an identity reference rather than a structural guide.

### Phase 8 — Role-Based Prompting (Key Breakthrough)

**Solution:** Strict role assignment for every input image.

| Input | Assigned Role |
|---|---|
| Garment Reference | Clothing — match exactly |
| Model Image | Identity — face and body |
| Pose Image | Structure only — do NOT copy identity |
| Background Image | Environment only |

Added explicit instruction: "*DO NOT use the person from the pose image.*"
**Result:** Identity bug resolved. Pose accuracy reached ~75–85%.

### Phase 9 — Production Pipeline

Full architectural rebuild into a modular, multi-stage pipeline. Moved from Streamlit to a React 18 + TypeScript + Vite web application with storyboard management, asset library, prints, multi-angle, and performance tracking.

---

## 5. Production Architecture

```
┌──────────────────────────────────────────────────────┐
│                   BotStudioX UI                      │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌────────┐  │
│  │ Generate │ │  Prints  │ │  Assets  │ │ Saved  │  │
│  │   Tab    │ │   Tab    │ │   Tab    │ │  Tab   │  │
│  └──────────┘ └──────────┘ └──────────┘ └────────┘  │
│  ┌──────────────────────────────────────────┐        │
│  │           Storyboard Library             │        │
│  └──────────────────────────────────────────┘        │
└──────────────────────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────────────────────┐
│                Pipeline Layer (pipeline.ts)          │
│                                                      │
│  planLookFromGarment()  →  AI Look Planner           │
```

```
│  generateFinalPrompt()  →  Prompt Builder            │
│  buildGarmentReferencePrompt()  →  Garment Ref Gen   │
│  buildCompositePrompt()  →  Final Composite Gen      │
│  buildRetryCompositePrompt()  →  Retry / Correction  │
│  buildMultiAnglePrompt()  →  Multi-Angle Gen         │
│  compositeDesignOnGarment()  →  Print Application    │
└──────────────────────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────────────────────┐
│              Gemini API Layer (gemini.ts)            │
│                                                      │
│  gemini-2.0-flash-preview  →  Planning / Text        │
│  gemini-2.0-flash-preview-image-generation           │
│                           →  All image generation    │
└──────────────────────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────────────────────┐
│                 Storage Layer                        │
│                                                      │
│  localStorage   →  Storyboard configs & metadata     │
│  IndexedDB      →  Saved image gallery records       │
└──────────────────────────────────────────────────────┘
```

### Runtime State per Storyboard

Each active storyboard maintains a `StoryboardRuntime` object in memory:

```
type StoryboardRuntime = {
  garmentDataUrls: string[];       // uploaded garment images
  modelDataUrls: string[];         // uploaded model images
  poseDataUrls: string[];          // uploaded pose images
  backgroundDataUrls: string[];    // uploaded background images
  lastPlan: LookPlan | null;       // AI-generated styling plan
  lastFinalPrompt: string | null;  // assembled generation prompt
  resultDataUrl: string | null;    // last output image
  timingsMs: PipelineTimings | null;
};
```

---

## 6. Pipeline — Step-by-Step Flow

```
User Inputs
    │
    ▼
[Step 1] AI Look Planning  ──────────────────────────── planLookFromGarment()
         Model: gemini-2.0-flash-preview
         Input: Garment image + user config overrides
         Output: LookPlan { occasion, style_keywords,
                            background_theme, model_pose,
```

```
                            footwear, accessories, ... }
    │
    ▼
[Step 2] Final Prompt Generation ──────────────────── generateFinalPrompt()
         Combines: LookPlan + user inputs + presets
         Output: Structured text prompt for composite generation
    │
    ▼
[Step 3] Garment Reference Generation ─────────────── buildGarmentReferencePrompt()
         Model: gemini-2.0-flash-preview-image-generation
         Input: Garment image(s)
         Output: Clean, standardized garment reference image
         Why: Removes noise, normalizes garment for better compositing
    │
    ▼
[Step 4] Final Composite Generation ────────────────── buildCompositePrompt()
         Model: gemini-2.0-flash-preview-image-generation
         Input order (critical):
           1. Garment Reference  → role: clothing
           2. Model Image        → role: identity
           3. Pose Image         → role: structure only
           4. Background Image   → role: environment
         Output: 1080×1440 px full-body composite image
    │
    ▼
[Step 5a] Multi-Angle Generation (optional) ─────────── buildMultiAnglePrompt()
          Input: Front composite + original garment ref
          Output: Side view + Back view images

[Step 5b] Print Application (optional) ──────────────── compositeDesignOnGarment()
          Input: Garment template + print design
          Method: Canvas API with multiply blending
          Output: Front / Back / Side print composites
```

### Hard Rules Applied in Every Composite Prompt

| Rule | Value |
|---|---|
| Framing | Full-body head-to-toe; 3:4 portrait; 1080×1440 px |
| Model Age | Young adult 18–23; must appear clearly adult |
| Garment Fidelity | Match silhouette, neckline, print, logos, seams exactly |
| Background Fidelity | If background image provided, match it closely |
| Global Avoid | Cropped limbs, extra people, deformed hands, text overlays, watermarks, added clothing layers, CGI look, minors |

---

## 7. Feature Modules

### 7.1 Generate Tab

The primary workflow. Users configure a storyboard, upload assets, and run the full pipeline. Includes:

- Storyboard config form (occasion, style, model, pose, background presets + free text overrides)
- Run / Retry controls
- Output preview with download
- Performance timing display

### 7.2 Prints Tab

Apply custom design prints onto garment template images.

- Upload base garment template (front / back / side)
- Upload print design image or specify a hex color
- Select garment type (T-shirt, Shirt, Hoodie, Saree, etc.) and gender
- Output: composite print images for all uploaded sides
- Method: Canvas API `multiply` blend mode for realistic ink-on-fabric simulation

### 7.3 Multi-Angle Tab

Generate additional views from a front composite.

- Input: existing front composite + garment reference
- Output: side view and back view
- Use case: e-commerce product pages requiring 360° coverage

### 7.4 Assets Tab

Manage reusable assets across storyboards:

- Model images (with ethnicity and theme metadata)
- Pose images
- Background images
- Tag-based search and filtering

### 7.5 Saved Images Gallery

All generated outputs are saved to IndexedDB for offline access.

- Thumbnail grid view
- Full-size image modal
- Download and delete controls
- Filtered by storyboard or all images

### 7.6 Storyboard Library

Manage multiple garment campaigns as separate storyboards.

- Create, rename, delete storyboards
- Each storyboard holds its own config, assets, and results
- Persisted to `localStorage`

---

## 8. Technical Stack

| Layer | Technology |
|---|---|

| | |
|---|---|
| Frontend Framework | React 18 + TypeScript |
| Build Tool | Vite |
| Styling | Custom CSS (design system in `/design-system`) |
| AI Model — Planning | Google Gemini 2.0 Flash Preview |
| AI Model — Image Generation | Google Gemini 2.0 Flash Preview (Image Gen) |
| Image Compositing | Canvas API (browser-native) |
| Local Storage | `localStorage` (storyboard configs) + `IndexedDB` (images) |
| Pose Detection (early) | OpenCV + MediaPipe (prototype phase) |
| Deployment | Static site via Vite build → `/docs` folder (GitHub Pages) |

### Key Source Files

| File | Responsibility |
|---|---|
| `src/lib/pipeline.ts` | Full pipeline: planning, prompts, composite, retry, multi-angle |
| `src/lib/storyboards.ts` | Storyboard data model, CRUD, localStorage persistence |
| `src/lib/gemini.ts` | Gemini API client — text and image generation |
| `src/lib/presets.ts` | All preset definitions (occasion, style, pose, model, footwear) |
| `src/lib/backgroundLibrary.ts` | Background template library |
| `src/lib/indexeddb.ts` | Saved images gallery (IndexedDB) |
| `src/App.tsx` | Root component — state orchestration, pipeline execution |
| `src/components/StoryboardFormCards.tsx` | Configuration form UI |
| `src/components/PrintsTab.tsx` | Prints pipeline UI |
| `src/components/MultiAngleTab.tsx` | Multi-angle generation UI |
| `src/components/AssetsTab.tsx` | Asset library UI |
| `src/components/SavedImagesPane.tsx` | Saved gallery UI |

---

## 9. Performance Benchmarks

Timings tracked per pipeline stage (approximate, depends on Gemini API response time):

| Stage | Typical Time |
|---|---|
| AI Look Planning | 3–6 seconds |
| Final Prompt Assembly | < 1 second (local) |

| | |
|---|---|
| Garment Reference Generation | 8–15 seconds |
| Final Composite Generation | 10–20 seconds |
| Multi-Angle (side + back) | 15–30 seconds |
| **Total (full pipeline)** | **25–50 seconds** |

### Quality Benchmarks (Current Production System)

| Metric | Status | Score |
|---|---|---|
| Pose Accuracy | Strong | ~80–85% |
| Identity Preservation | Strong | Consistent |
| Garment Transfer Fidelity | Good | Reliable |
| Background Matching | Good | Reliable |
| Output Stability (run-to-run) | High | Improved significantly |
| Overall UX | Advanced | Multi-tab, storyboard system |

---

## 10. Known Limitations

| Limitation | Root Cause | Workaround |
|---|---|---|
| Pose accuracy not 100% | Gemini is generative, not a controlled diffusion model | Use OpenCV skeleton for better signal |
| Finger / hand artifacts | Hand generation is a known challenge for image models | Crop or select outputs with clean hands |
| Cloth physics imperfect | No physics simulation; Gemini infers drape | Use high-quality garment reference images |
| Output variability | Gemini has inherent stochasticity | Use Retry feature; run multiple times |
| No real-time preview | Generation takes 25–50 seconds | Not applicable for real-time use cases |
| Garment type constraints | Works best for standard apparel; complex traditional wear (saree draping) less accurate | Provide clear multi-angle garment references |

---

## 11. Glossary

| Term | Definition |
|---|---|
| Composite Image | Final output image with model, garment, pose, and background combined |
| Garment Reference | Intermediate clean image of the garment generated before the final composite |

| | |
|---|---|
| LookPlan | AI-generated JSON object describing occasion, style, background theme, accessories, and pose for a given garment |
| Role-Based Prompting | Technique of assigning explicit roles (identity, structure, clothing, environment) to each input image in the Gemini prompt |
| Storyboard | A named project unit containing configuration, assets, and generated results for one garment campaign |
| StoryboardRuntime | In-memory state object tracking current uploaded assets and generation results for an active storyboard |
| Input Ordering | The sequence in which images are passed to Gemini — critical for controlling how the model interprets each image |
| Multiply Blend | Canvas API blending mode used in the Prints pipeline to simulate design ink on fabric texture |
| IndexedDB | Browser-native key-value store used to persist generated image data locally without a backend |
| Pose Skeleton | Stick-figure representation of a pose extracted by OpenCV + MediaPipe; used as a clean pose signal |

---

*BotStudioX — Built by Mohan. From prototype to production AI fashion platform.*